Aperture — Frontend Architecture

Version: v1.0

Document Type: Frontend Architecture

Related Documents

- Product Requirements Document
- Technical Architecture
- API Specification
- Design System (Future)

---

1. Purpose

This document defines the architectural principles used to build Aperture's frontend.

It focuses on:

- Application structure
- Component organization
- State management
- Routing
- Responsive design
- Performance
- Accessibility
- Visual consistency

It intentionally avoids implementation-specific details that belong in code.

---

2. Frontend Philosophy

The frontend should embody Aperture's brand identity while remaining maintainable and performant.

The guiding principles are:

- Content-first
- Cinematic, not flashy
- Consistent interaction patterns
- Reusable components
- Responsive by default

- Accessibility from the beginning

Every UI decision should improve content discovery rather than distract from it.

---

3. Technology Stack

Core technologies:

- Next.js
- React
- TypeScript
- Tailwind CSS
- Framer Motion
- TanStack Query
- Zustand

Each technology serves a distinct purpose and should not overlap responsibilities.

---

4. Application Architecture

The frontend is divided into feature-based domains rather than page-based folders.

```
src/
├── app/
├── components/
├── features/
├── hooks/
├── services/
├── lib/
├── layouts/
├── styles/
├── types/
└── utils/
```

This organization allows features to evolve independently while sharing common infrastructure.

---

5. Routing Strategy

Next.js App Router provides:

- Nested layouts
- Server Components where appropriate
- Dynamic routes
- Loading states
- Error boundaries

Primary routes include:

- Home
- Movie
- TV Show
- Person
- Search
- Profile
- Lists
- Review
- Settings
- Administration

Routing should mirror the application's domain model rather than backend implementation.

---

6. Component Philosophy

Components should be:

- Small
- Reusable
- Predictable
- Composable

Prefer composition over inheritance.

Business logic should remain outside presentational components whenever practical.

---

7. Component Hierarchy

The UI is organized into layers.

Layout Components

Examples:

- Navigation
- Sidebar
- Footer
- Page Shell

Feature Components

Examples:

- Movie Card
- Review Card
- List Card
- Recommendation Carousel

Shared Components

Examples:

- Button
- Badge
- Avatar
- Modal
- Dialog
- Tabs
- Pagination

Primitive Components

Small reusable building blocks.

Examples:

- Icon
- Divider
- Skeleton
- Spinner

This hierarchy minimizes duplication and promotes consistency.

---

8. Design System

Aperture uses a custom design system.

Core elements include:

Typography

Spacing

Color palette

Border radius

Elevation

Animation timing

Icons

These design tokens should be centralized rather than scattered throughout the application.

---

9. Responsive Design

Responsive behavior should be intrinsic rather than implemented page by page.

Guiding principles:

- Mobile-first layouts
- Flexible grids
- Relative sizing
- Container-based layouts
- Fluid typography
- Adaptive spacing

Avoid designing for specific device models.

Instead, components should respond naturally to available space.

---

10. State Management

State is categorized by ownership.

Server State

Managed by TanStack Query.

Examples:

- Movie metadata
- Reviews
- Search results
- Discovery feeds

Client State

Managed by Zustand.

Examples:

- Theme
- Sidebar state
- Dialog visibility
- Temporary filters

Local Component State

Managed with React state.

Examples:

- Form inputs
- Hover interactions
- Temporary UI state

Choosing the correct state location reduces unnecessary complexity.

---

11. Data Fetching

The frontend communicates exclusively through the REST API.

Guidelines:

- Use TanStack Query for server data
- Cache responses appropriately
- Prefetch predictable navigation targets
- Invalidate caches after mutations
- Handle loading and error states consistently

Components should never perform direct data fetching if an existing feature hook already encapsulates that behavior.

---

12. Performance Strategy

Performance should be considered from the beginning.

Key techniques include:

- Server-side rendering for content pages
- Lazy loading below-the-fold content
- Dynamic imports for heavy components
- Image optimization
- Route prefetching
- Code splitting
- Memoization only when profiling justifies it

Optimization should target measurable bottlenecks.

---

13. Animation Philosophy

Animation should reinforce interaction, not distract from it.

Suitable uses:

- Page transitions
- Card hover states
- Modal transitions
- Skeleton loading
- Expand/collapse interactions

Avoid excessive animation that delays user interaction.

Motion should remain subtle, smooth, and purposeful.

---

14. Accessibility

Accessibility is a core requirement.

The frontend should support:

- Semantic HTML
- Keyboard navigation
- Focus management
- Screen readers
- Adequate color contrast
- Reduced motion preferences

Accessibility should be verified throughout development rather than retrofitted later.

---

15. Error Handling

Every feature should gracefully handle:

- Loading
- Empty state
- Error state
- Partial content

Users should always understand what happened and what they can do next.

---

16. Theming

Aperture supports:

- Dark theme (default)
- Light theme

Future:

- Dynamic accent colors
- Seasonal themes
- Accessibility themes

Themes should be implemented using design tokens rather than duplicated styles.

---

17. Asset Management

Static assets include:

- Logos
- Icons
- Illustrations
- Local images

Remote assets include:

- Posters
- Backdrops
- Profile images

Remote media should leverage Next.js image optimization where supported.

---

18. Testing Strategy

Frontend testing should include:

Unit Tests

- Utility functions
- Custom hooks
- Pure components

Component Tests

- Rendering
- User interactions
- Accessibility

End-to-End Tests

Critical user journeys:

- Login
- Search
- Review submission
- List creation
- Discovery browsing

19. Future Evolution

Potential enhancements:

- Progressive Web App
- Offline support
- Mobile application
- Multi-language interface
- Component documentation with Storybook
- Design token synchronization

The architecture should accommodate these additions without major restructuring.

---

20. Interview Discussion Points

Be prepared to explain:

- Why Next.js over a client-side SPA?
- Why separate server state from client state?
- Why use feature-based organization instead of page-based folders?
- How do you design responsive layouts without targeting specific devices?
- How do animations improve usability rather than aesthetics?
- Why build a custom design system instead of relying entirely on a UI framework?

---

21. Summary

The Aperture frontend is designed as a scalable, component-driven application that balances aesthetics with engineering discipline.

By separating presentation, state, and business concerns while adopting a reusable design system and responsive architecture, the frontend provides a strong foundation for both rapid feature development and long-term maintainability.